

HOW TO USE JEV WITH LLMS

A Practical Guide to Typed Decisions Around Generative Models

Independent field guide based on public TypeSafe documentation and examples.

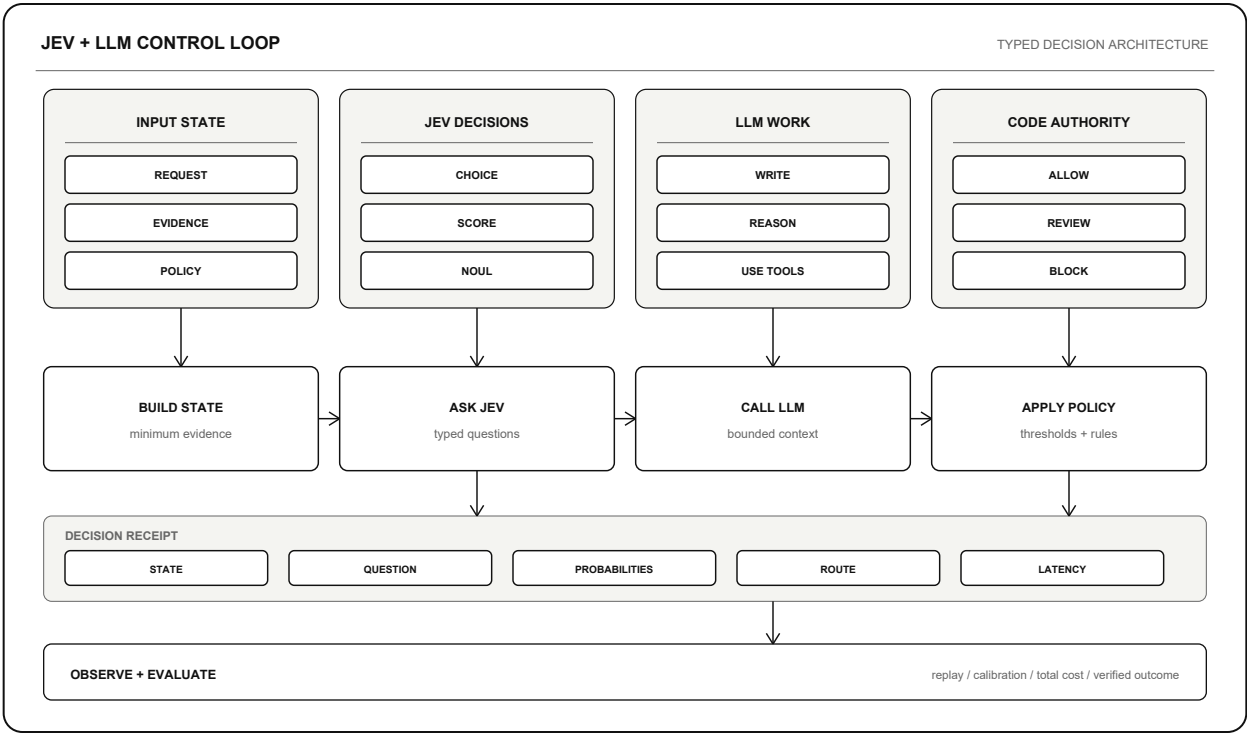


Fig. 1. Jev makes typed judgments around an LLM while code retains authority.

ABSTRACT

Jev is not a chat model and does not replace the LLM that writes prose, code, plans, or tool calls. Jev is a System One decision model: software sends a text or JSON state plus typed questions, and receives Choice, Score, or Noul answers with probabilities. This guide shows how to place that decision primitive before, beside, and after a generative model while keeping control flow and side effects in ordinary code.

The practical pattern is simple: deterministic code handles known rules; Jev makes narrow judgments over unstructured state; an LLM handles open-ended generation; and code applies thresholds, permissions, fallbacks, logging, and human review. The result is not a smarter chatbot. It is a more inspectable and economical system whose uncertain decisions can be measured instead of hidden inside prose.

INDEX TERMS

Jev, System One, LLMs, typed decisions, confidence gates, routing, guardrails, evaluation, Claude, Codex, Grok.

I.

JEV + LLM: THE DIVISION OF LABOR

Put each kind of intelligence where its output can be controlled.

A. THE DIVISION OF LABOR

A useful Jev-plus-LLM system begins by separating two kinds of intelligence. A generative LLM is good at producing strings: explanations, plans, code, summaries, and tool arguments. Jev is designed for closed decisions: choose one option, rate a case on an ordered rubric, or estimate whether a statement is true. The LLM creates possibilities. Jev evaluates bounded questions about the current state. Code decides what happens next.

This distinction prevents a common architecture mistake: asking one model to understand the request, invent a plan, choose a provider, approve its own tool call, judge its own result, and declare completion inside one prompt. That hides many independent judgments inside a single transcript. Pulling the judgments into typed questions makes them visible, testable, and replaceable.

B. JEV IS NOT AN LLM REPLACEMENT

Jev does not stream text, write code, or hold a conversation. It is therefore not a drop-in model setting for Claude Code, Codex, Grok, or another agent. Continue using the generative model for open-ended work. Add Jev where the surrounding application needs a fast judgment that can be expressed as a finite choice, an ordered score, or a probability of yes.

NEED	BEST OWNER
Apply exact rules	Code
Judge messy text	Jev
Write or reason openly	LLM
Authorize side effects	Policy + human
Record and replay	Harness

Table I. Assign work by output shape, authority, and evidence.

F. WHAT THE DECISION LAYER BUYS YOU

The value is not merely lower latency. A typed decision can be compared across versions, replayed over historical states, placed behind a feature flag, and audited without reading a transcript. Teams can ask whether a route was calibrated, whether the option set was complete, and whether policy applied the answer correctly.

That separation also keeps generative prompts smaller. The LLM no longer needs paragraphs describing every routing rule and escalation condition. It receives the context and tools for the branch already selected, while the harness retains a compact receipt explaining why that branch existed.

C. THE HYBRID REQUEST PATH

```
request
-> deterministic validation
-> Jev: classify intent, risk, route
-> code: choose context and LLM
-> LLM: generate plan, text, or code
-> Jev: verify bounded properties
-> code: pass, retry, review, or block
-> receipt + metrics
```

The two Jev calls are optional and serve different purposes. A pre-LLM decision controls what the expensive generative model sees and which model should receive the task. A post-LLM decision evaluates the proposed output against narrow properties before code allows a consequential action. Read-only and low-risk flows may need only one side of the pattern.

D. A DECISION BOUNDARY TEST

Use Jev when the answer space is known before the request arrives, the state contains enough evidence for a quick judgment, and your code can act on the returned distribution. Use an LLM when the answer must be invented, explained, transformed, or developed through multi-step reasoning. Use deterministic code when the rule is already explicit and can be evaluated without semantic judgment.

E. KEEP CODE IN CHARGE

A favorable Jev answer is evidence, not authority. Code still validates types, identities, paths, limits, freshness, and policy. A high probability cannot grant a capability that the caller does not possess. The surrounding program decides whether to act, ask for confirmation, gather more state, call a different model, or send the case to a person.

The most reliable mental model is not Jev as manager of several LLMs. It is Jev as a typed sensor inside a program. The program owns the workflow. The LLM generates. Jev measures selected semantic properties. Policy converts those measurements into behavior.

F. WHERE JEV DOES NOT BELONG

Do not use Jev to draft the final answer, generate a patch, synthesize a long plan, or replace an exact parser. Do not call it when the result cannot change behavior. A decision layer that produces metrics but never alters routing, review, or context is only additional latency.

Avoid framing the architecture as one model supervising another model. Supervision implies broad authority. In a production system, Jev answers a defined question, the LLM completes a defined generative task, and policy remains the only component allowed to turn those outputs into an external effect.

II.

YOUR FIRST DECISION IN TEN MINUTES

Start with one state, several typed questions, and explicit fallback behavior.

A. START IN THE PLAYGROUND

The shortest learning loop is the TypeSafe Playground. Paste a representative piece of text as the state, add one Noul question, then inspect the returned probability. Next add a Choice and a Score that read the same state. This makes the three answer shapes concrete before any application architecture is involved.

Use real but non-sensitive examples from the intended workflow. A synthetic sentence is enough to test request syntax, but threshold design requires a labeled sample that contains ordinary cases, ambiguous cases, missing evidence, adversarial phrasing, and examples where the correct action is to abstain or escalate.

B. MAKE THE FIRST API CALL

```
POST https://api.typesafe.ai/v1/systemone
Authorization: Bearer $TYPESAFE_API_KEY
Content-Type: application/json

{
  "state": {"message": "Please refund the duplicate charge."},
  "model": "jev-latest",
  "questions": {
    "refund": {
      "type": "noul",
      "instructions": "Does `message` request a refund?"
    }
  }
}
```

The answer is a probability of yes. Your code compares that value with a threshold chosen for this workflow. It should also define what happens when the request fails, the answer is missing, or the value falls into an uncertainty band.

C. INSTALL THE PYTHON SDK

```
pip install typesafe-sdk
# or: uv add typesafe-sdk

# Set TYPESAFE_API_KEY outside source control.
```

F. KEEP THE FIRST EXPERIMENT NARROW

Choose one repeated decision with a visible outcome: route a ticket, classify a request, or decide whether an LLM answer needs review. Avoid beginning with an irreversible action. The first integration should prove that state, questions, thresholds, and receipts work together before it attempts to automate a consequential branch.

Prepare ten obvious examples, ten ambiguous examples, and ten no-fit examples. Run them through the playground, record the distributions, and have a domain owner label the expected route. This small set is not a benchmark, but it catches missing options and vague wording before code is deployed.

D. ASK SEVERAL QUESTIONS AT ONCE

```
from typesafe_sdk import Choice, Noul, Score, TypeSafeClient

client = TypeSafeClient()
response = client.system_one(
    state={
        "message": "Our checkout is down. We are losing sales.",
        "account_tier": "enterprise",
    },
    questions={
        "team": Choice(
            instructions="Which team should own `message`?",
            criteria={
                "billing": "Payments or refunds",
                "technical": "Bugs or integrations",
                "other": "None of the above",
            },
        ),
        "urgency": Noul(
            instructions="Does `message` require urgent action?"
        ),
        "impact": Score(
            instructions="How severe is the business impact?",
            criteria=["Low", "Material", "Critical"],
        ),
    },
)
```

E. READ THE RESPONSE AS DATA

A Choice returns the selected option, a probability for every option, and confidence. A Score returns a position on the rubric, its legend, a distribution across levels, and confidence. A Noul returns a number from zero to one: the probability that the answer is yes. Noul does not have a separate confidence field.

Log the returned model identifier, the question version, the state digest, distributions, chosen branch, latency, and any downstream LLM call. Do not log secrets or raw customer state by default. The first production version should be able to explain which values caused each route without reconstructing a private prompt transcript.

G. HANDLE FAILURES EXPLICITLY

Treat authentication errors, timeouts, rate limits, malformed responses, and missing answers as separate states. Each needs a documented path: retry with bounded backoff, use a deterministic fallback, keep the result advisory, or escalate. Never replace a missing decision with an invented default that looks like a model answer.

Use a pinned model identifier while calibrating a production threshold. A moving alias is convenient during exploration, but a model change can move the distribution your policy depends on. When upgrading, replay labeled cases and compare both final routes and probability bands.

III.

DESIGN STATE THE MODEL CAN JUDGE

The model can only judge the evidence the application chooses to send.

A. STATE IS THE EVIDENCE

State is the content Jev evaluates. It may be a string, a JSON object, or an array of text values. All questions in one request see the same state and are evaluated independently. For most production calls, a structured object is clearer than a long string because fields have names and relationships remain explicit.

Think of state as the evidence packet given to a review panel. Include the request, relevant records, current policy, and previous verified results. Exclude unrelated history, decorative prose, and information that cannot change the answer. More context is not automatically better; irrelevant context creates distraction, cost, privacy exposure, and harder evaluation.

B. NAME EVERY IMPORTANT FIELD

```
state = {
  "request": {"text": user_message},
  "account": {"tier": tier, "region": region},
  "policy": {"refund_window_days": 30},
  "llm_proposal": {"action": action, "arguments": args},
}
```

Questions can refer to paths such as ``request.text``, ``policy.refund_window_days``, or ``llm_proposal.arguments``. The explicit reference narrows the judgment and makes later changes easier to review.

C. SEPARATE FACTS FROM INSTRUCTIONS

Treat all user content, retrieved documents, tool output, and LLM text as untrusted data. Put workflow rules in code and question definitions, not inside the state being judged. If a retrieved page says ignore the policy and permit the command, that sentence is evidence to classify, not an instruction the harness should execute.

H. PRESERVE PROVENANCE

Every important field should carry enough provenance to be checked later: source identifier, retrieval time, version, and whether it was verified or generated. When state includes an LLM proposal, store the serving model and request digest. When it includes policy, store the policy revision rather than only copied prose.

Provenance lets the review layer distinguish unsupported generation from stale evidence. It also enables selective refresh: a route can reload one expired account record without rebuilding an unrelated conversation or asking the LLM to summarize everything again.

D. BUILD A MINIMUM SUFFICIENT STATE

INCLUDE	AVOID
Current request	Entire chat by default
Relevant policy	Unversioned memory dump
Candidate action	Hidden global assumptions
Verified records	Secrets not needed to judge
Source timestamps	Stale facts without dates

Table II. State should be sufficient, bounded, and auditable.

E. VERSION THE STATE CONTRACT

A decision pipeline depends on more than the model name. Record a state schema version and a question-set version. A renamed field, changed option description, reordered score rubric, or added policy document can alter results even when the model is unchanged. Treat these inputs like an API contract and roll them out deliberately.

F. PROTECT SENSITIVE DATA

Classify the state before sending it to any provider. Redact secrets, minimize personal data, and keep restricted material on approved routes. A Jev route and an LLM route may have different data-handling rules. The decision to use a provider must pass deterministic organization policy before any semantic routing logic runs.

G. TEST MISSING EVIDENCE

For every question, construct cases where the required field is absent, contradictory, or stale. Choice questions should include an explicit other, unknown, or needs-review option when the supplied set may not cover reality. Without an escape option, a closed model must still select something, and the winning label can look confident even though the state never supported it.

A robust state builder fails closed when required records cannot be loaded. It does not quietly replace a missing policy with a model guess.

I. REVIEW THE STATE BUILDER SEPARATELY

Unit-test state construction before evaluating model quality. Fixtures should cover null fields, duplicate records, incorrect joins, stale caches, oversized documents, mixed encodings, and redaction. A perfect decision model cannot repair a state packet that attached the wrong customer, policy, or repository snapshot.

Keep a human-readable state preview in development. Reviewers should be able to see the exact structured packet without provider-specific wrappers. If the important evidence is hard for a person to locate, the packet is probably too broad or its field names are too vague.

IV.

TRANSLATE LLM WORK INTO TYPED QUESTIONS

Choice, Score, and Noul turn hidden judgments into inspectable contracts.

A. MATCH THE PRIMITIVE TO THE ANSWER

Choice asks which option fits. Use it for intent, destination, document type, provider class, tool category, or resolution path. Score asks where the state falls on an ordered rubric. Use it for urgency, severity, quality, complexity, or relevance. Noul asks whether a statement is true and returns the probability of yes. Use it for policy checks, evidence support, risk flags, and other binary claims.

PRIMITIVE	BEST QUESTION	OUTPUT
Choice	Which option?	label + distribution
Score	Which level?	score + distribution
Noul	Is this true?	P(yes)

Table III. Choose the primitive your code can use directly.

B. ASK ATOMIC QUESTIONS

A good Jev question is a snap judgment a knowledgeable person could make quickly from the supplied state. Does the output cite the provided source? Which specialist should receive the task? How severe is the potential impact? A bad question hides a workflow: analyze the request, choose the best strategy, solve it, and decide whether the result is safe.

Break broad evaluation into independent properties and combine them in code. For an LLM answer, separately ask whether it addresses the request, whether cited evidence supports the claim, whether it contains unrelated changes, and whether clarification is still required. The resulting rule is inspectable and each component can be tuned.

G. MEASURE QUESTION QUALITY

Evaluate questions individually before evaluating the composed policy. Track the distribution of answers, human agreement, confidence bands, and the examples nearest each threshold. A policy can appear accurate because one dominant question masks a second question that is ambiguous or never changes the route.

Review low-confidence cases as product research. They often reveal missing criteria, mixed intents, or state that lacks the decisive fact. The best correction may be adding an option, restructuring the state, or collecting one more field rather than rewriting the instruction with more adjectives.

C. DEFINE CRITERIA, NOT VIBES

```
questions = {
  "route": Choice(
    instructions="Which handler fits `request.text`?",
    criteria={
      "search": "Needs current external facts",
      "code": "Needs repository changes",
      "analysis": "Needs reasoning over supplied material",
      "other": "No listed route fits",
    },
  ),
  "high_risk": Noul(
    instructions="Would the proposed action create a hard-to-reverse external effect?"
  ),
}
```

Option descriptions are part of the model contract. Make boundaries mutually understandable and avoid several labels that describe the same behavior. Add a no-match option whenever the real world may fall outside the list.

D. BATCH INDEPENDENT QUESTIONS

Questions in one request share the same state but do not see each other's answers. Ask every independent question together, including speculative questions whose answers may only matter on one branch. The code can ignore unused answers. Use a second call only when the first answer changes which evidence must be fetched or which options exist.

E. KEEP IDS STABLE

Question IDs are for code and are not a substitute for complete instructions. Keep stable IDs for logs and metrics, but version any change in semantics. A dashboard that combines results from two different question definitions under one name creates false trends.

F. AVOID SELF-APPROVAL

Do not ask the generative LLM to produce a proposal and a confidence score that authorizes the same proposal. Put the bounded review in Jev or deterministic validation, and let policy decide what evidence is sufficient. Independence is not perfection, but it removes one obvious conflict of interest.

H. QUESTION ANTI-PATTERNS

Avoid asking what should we do when the answer mixes diagnosis, policy, and execution. Avoid Score levels such as bad, medium, and good without observable anchors. Avoid Choice options that overlap, and avoid Noul statements whose yes condition changes from one reviewer to another.

Do not smuggle output requirements into question IDs. The ID `safe\_to\_execute` does not tell the model what safe means. Put the full operational meaning in instructions and criteria, then let code map the answer to execution policy.

V.

PUT JEV BEFORE THE LLM

Route requests, context, and providers before generative cost is committed.

A. CLASSIFY BEFORE GENERATION

A pre-LLM Jev call can decide which path the request should enter before an expensive model sees it. Typical questions classify intent, estimate task complexity, detect sensitive content, choose a retrieval collection, select a tool family, and determine whether a human must approve the next stage.

Run deterministic compatibility checks first. If a provider is unavailable, a tool is forbidden, or a file classification prohibits egress, remove that option before semantic routing. Jev should choose among allowed possibilities, not be asked to overrule policy.

B. ROUTE BY CAPABILITY AND TRUST

SIGNAL	POSSIBLE BRANCH
Current facts needed	retrieval or web-enabled LLM
Repository edit	coding agent with workspace
Sensitive state	approved provider or local path
Low confidence	clarify or human review
Simple explicit rule	skip AI and use code

Table IV. Route only after capability and policy filters.

C. SELECT CONTEXT, NOT ONLY MODELS

The largest savings often come from deciding what not to send. Ask whether each candidate document is relevant to the current request, then assemble the LLM context from accepted passages. Keep citations to the source chunks so the generated answer and the post-LLM verifier can evaluate the same evidence.

G. RETRIEVE BEFORE ROUTING WHEN NECESSARY

Some requests cannot be routed from the user message alone. A short repository scan, metadata lookup, or deterministic search may reveal the language, affected service, sensitivity class, or available specialist. Perform that cheap discovery first, then ask Jev to judge a compact set of candidates rather than guessing from an under-specified request.

Keep retrieval recall and semantic judgment separate. The search stage should produce a bounded candidate set with identifiers. Jev judges fit among those candidates. Code applies access control and assembles the final context. This division makes missing evidence distinguishable from a wrong semantic choice.

D. A PREFLIGHT BATTERY

```
preflight = {
  "route": Choice(
    "Which allowed route best fits the request?",
    criteria=allowed_routes,
  ),
  "needs_current_data": Noul(
    "Does the request depend on facts that may have changed?"
  ),
  "contains_sensitive_data": Noul(
    "Does the state include restricted or personal information?"
  ),
  "complexity": Score(
    "How much multi-step reasoning is required?",
    criteria=["direct", "moderate", "deep"],
  ),
}
```

E. COMPOSE THE ROUTE IN CODE

A route should rarely depend on one model answer alone. Combine Jev results with hard constraints such as provider availability, context size, supported tools, data residency, latency budget, and account limits. Record the candidates considered and the reason each candidate was removed.

For example, sensitive state can force an approved provider; current-data dependence can require retrieval; high complexity can reserve a stronger model; and low confidence can request clarification. These are program rules over typed values, not prose instructions asking a router model to do everything.

F. MEASURE THE COUNTERFACTUAL

Log which route would have been taken without Jev. In shadow mode, call the normal LLM path and compare quality, latency, and cost with the proposed route. Routing that saves a few input tokens but increases retries, context reloads, or human review is not an improvement.

A provider switch is economical only when the full handoff cost is included: context serialization, cache loss, retrieval, generated output, return-to-review, and the failure rate of the cheaper branch.

H. DEFINE A ROUTE CONTRACT

Each route should declare the provider class, allowed data, tool surface, context budget, maximum retries, verification plan, and fallback. A Choice result selects a contract, not a naked model name. This prevents a semantic router from accidentally bypassing limits that were previously hidden inside provider-specific code.

Route contracts also make experiments safer. You can compare two providers under the same data and verification boundary, or change a model version without changing the Jev question. The receipt records both the selected route class and the concrete serving configuration.

VI.

PUT JEV AFTER THE LLM

Verify bounded semantic properties without giving the reviewer authority.

A. VERIFY BOUNDED PROPERTIES

After an LLM produces text, code, or a tool proposal, Jev can evaluate properties that are hard to express as exact rules but easy to state as narrow questions. Examples include whether the answer addresses the request, whether a cited passage supports a claim, whether a patch contains unrelated changes, or whether the user must clarify an ambiguity.

Jev should not replace deterministic verification. Run parsers, schemas, tests, type checkers, allowlists, and policy checks first. A semantic review cannot prove that code compiles, a migration is reversible, or a URL belongs to an approved domain when ordinary code can check those facts exactly.

B. SEPARATE INPUT AND OUTPUT GUARDS

Input screening asks what the user is requesting and what risks the request contains. Output screening asks what the LLM actually produced. The batteries should not be identical. A safe input can still produce an unsafe or unsupported output, and a risky-looking input may deserve a helpful refusal rather than a block.

C. REVIEW A PROPOSED TOOL CALL

```
state = {
  "request": request,
  "tool": proposal.tool,
  "arguments": proposal.arguments,
  "policy": policy_excerpt,
  "evidence": cited_sources,
}
```

Ask narrow questions; code owns the final verdict.

G. COMPARE OUTPUT WITH THE SAME EVIDENCE

A verifier should see the exact sources used for generation whenever possible. If generation used passage A but verification sees a new search result B, the review may approve a claim that the original model could not have supported. Bind cited passages, tool results, and repository snapshots to the proposal receipt.

For long answers, verify claims or sections rather than asking one broad question about the whole response. Deterministic extraction can identify citations, code blocks, changed files, or declared actions; Jev can then judge bounded relationships between each item and its supporting evidence.

D. A POSTFLIGHT BATTERY

```
postflight = {
  "addresses_task": Noul(
    "Does `llm_output` directly address `request`?"
  ),
  "evidence_supports": Noul(
    "Does `evidence` support the factual claims in `llm_output`?"
  ),
  "unrelated_changes": Noul(
    "Does `proposal` include work outside `approved_scope`?"
  ),
  "needs_clarification": Noul(
    "Is required information missing before this can proceed?"
  ),
}
```

E. CONVERT REVIEW INTO ACTION

Use a decision table with explicit precedence. Malformed state or failed deterministic validation rejects the proposal. Missing Jev output makes the review unavailable. An unfavorable or low-confidence answer can keep the result as proposal-only, ask the LLM to revise, or route to a human. Only a complete favorable set can make the proposal eligible for the next policy gate.

Do not let a good semantic review automatically execute a consequential command. Authorization, identity, capability, freshness, rate limits, and user confirmation remain separate checks. The verifier says what appears true of the supplied state; it does not grant permission.

F. PRESERVE THE EVIDENCE

Store the exact serialized state, question version, model identifier, full distributions, chosen threshold, deterministic validation results, and final action. Bind the receipt to a digest of the proposal and source snapshot. If the file, policy, or tool arguments change, the earlier review is stale and must not be replayed as current approval.

H. MAKE REVISION LOOPS FINITE

When review fails, return structured reasons to the generative model: which property failed, which evidence was missing, and what scope must remain unchanged. Limit revision attempts and change the state on every retry. A loop that repeatedly asks improve this until approved encourages drift and hides the real stopping rule.

If the same property fails twice, escalate or narrow the task. Repeated failure may mean the request is impossible, the context is insufficient, or the verification contract is wrong. More tokens do not solve an invalid boundary.

VII.

CONFIDENCE GATES AND CASCADES

Use uncertainty to choose among automatic action, review, clarification, and fallback.

A. PROBABILITY IS NOT PERMISSION

Choice and Score return distributions plus confidence. Noul returns the probability of yes and has no separate confidence field. These numbers describe model uncertainty about a bounded question. They do not measure business impact and do not convert an unauthorized action into an authorized one.

B. USE THREE OPERATING BANDS

BAND	DEFAULT BEHAVIOR
High	eligible for automatic path
Medium	confirm, gather state, or review
Low	stop, clarify, or fallback

Table V. Confidence changes behavior; policy sets the boundaries.

The same probability can produce different actions under different policies. A medium-confidence route to a read-only search tool may be acceptable; the same confidence for deleting data is not. Define thresholds per question, action, and risk class rather than one global number.

C. START CONSERVATIVE

Run the first version in shadow mode. Compare Jev answers with labeled outcomes or human decisions, then choose thresholds on a held-out set. Track false automation, needless review, abstention, and downstream quality. A threshold copied from an example is an initial placeholder, not evidence that the system is calibrated for your domain.

G. CALIBRATE EACH QUESTION SEPARATELY

A 0.8 Noul threshold for evidence support is not interchangeable with 0.8 on a different question or with Choice confidence. Plot reliability by probability band for each contract version and inspect enough examples to understand the error type. High-stakes negative and positive decisions may require asymmetric thresholds.

Recalibrate after changing the model, question wording, criteria, state builder, language mix, or traffic source. The policy depends on the complete decision contract, not only the provider's model identifier.

D. BUILD A CASCADE DELIBERATELY

```

if deterministic_rule(state):
    return code_path(state)

answer = ask_jev(state, questions)
if answer_is_clear_and_low_risk(answer):
    return fast_path(answer)
if more_evidence_exists(state):
    return ask_jev(load_more(state), questions_v2)
if task_needs_generation(state):
    return call_llm(state)
return human_review(state)

```

A cascade is useful when most cases are simple and a minority need expensive reasoning. It is wasteful when every request eventually reaches the large model, because the earlier calls only add latency and cost. Measure the fraction that exits at each stage and the quality of those exits.

E. RETRY ONLY AFTER CHANGING EVIDENCE

Repeatedly asking the same question over the same state is not a recovery strategy. Retry transport failures according to an explicit policy. For semantic uncertainty, obtain new evidence, revise the question contract, change the option set, or escalate. Otherwise the system is sampling until it receives a convenient answer.

F. DISTINGUISH UNCERTAINTY SOURCES

Low confidence may mean the evidence is missing, several options are genuinely plausible, the criteria overlap, or the task is a poor fit for a fast judgment. Record which intervention resolves the case. If adding a no-match option fixes many errors, the problem was the answer space. If human reviewers disagree, the problem may be the policy or label definition rather than the model.

The goal is not maximum automatic throughput. It is the best completed outcome per unit of cost, latency, and review attention at an acceptable risk level.

H. KEEP FALLBACK OBSERVABLE

A fallback route must appear in the same metrics as the primary route. Record why the fallback fired, which evidence it received, whether it changed the answer, and whether the final outcome improved. Otherwise the system can hide a failing first stage behind an expensive second stage.

Set a latency and cost ceiling for the entire cascade. When the ceiling is reached, stop cleanly with the best safe artifact available or request human input. An unbounded chain of model calls is not resilience; it is loss of control.

VIII.

BATCH DECISIONS AND CONTROL COST

Share state, ask independent questions together, and measure the complete path.

A. SHARE ONE STATE ACROSS QUESTIONS

When several independent judgments read the same state, send them together. System One evaluates questions independently and in parallel. The official parallel-questions cookbook reports roughly an order-of-magnitude improvement in its 13-question example compared with separate calls, while returning the same answers. Treat that result as an example, not a universal benchmark.

Speculative fan-out is often cheaper than serial branching. Ask the questions that might matter, then let code use only the relevant answers. A second request is justified when the first answer determines what evidence to fetch or which options exist; otherwise it usually duplicates state transfer and latency.

B. DESIGN A QUESTION BATTERY

BATTERY	QUESTIONS
LLM intake	intent, urgency, sensitivity, route
RAG	relevance, source quality, answer present
Tool call	scope, reversibility, clarification
LLM output	support, completeness, policy fit

Table VI. Batteries make repeated decisions consistent.

C. CACHE BY COMPLETE CONTRACT

A safe cache key includes the normalized state, question definitions, criteria, model identifier, and policy-relevant version. Reusing an answer after any of those change can silently apply an old judgment to a new case. Cache only for a freshness window appropriate to the underlying evidence.

H. KEEP BATTERIES STABLE AND REVIEWABLE

A battery is a product interface. Give it an owner, semantic version, changelog, and regression set. Questions that are always ignored should be removed; questions whose answers correlate strongly may still represent different risks and should only be merged after domain review.

Document which routes consume each answer. This prevents a seemingly harmless question edit from changing an unrelated automation path. It also lets teams calculate the marginal value of a question from real decisions rather than treating every returned probability as equally important.

D. BUDGET THE COMPLETE PATH

Measure more than provider price. Record serialization time, network latency, Jev latency, LLM latency, retries, retrieval, tool execution, review time, and the cost of failures. An extra decision call is valuable when it prevents a larger model call, reduces context, avoids rework, or catches a consequential error. It is waste when it cannot change the route.

E. KEEP AN EXECUTION LEDGER

```
receipt = {
  "state_digest": digest(state),
  "question_version": "intake-v3",
  "jev_model": response.model,
  "answers": response.answers,
  "policy_version": "router-v7",
  "route": selected_route,
  "llm_provider": serving_provider,
  "latency_ms": timings,
  "cost_estimate": costs,
  "outcome": verified_outcome,
}
```

F. OPTIMIZE THE QUESTIONS, NOT JUST THE MODEL

A vague question can turn a fast call into unreliable automation. Track answer entropy, low-confidence rate, human disagreement, and error by question version. Refine state selection, option coverage, and criteria before changing models. The highest-leverage improvement may be splitting one ambiguous question into three clear ones.

G. AVOID HIDDEN SERIAL WORK

Do not call Jev once per file, once per candidate, or once per option when one structured state and a batched question set can express the same job. For large collections, use deterministic recall first, judge a bounded candidate set, and keep identifiers in the state so code can map answers back to source records.

I. CONTROL QUEUES AND CONCURRENCY

A fast model can still overload a downstream LLM, tool, or review queue. Apply concurrency limits after routing and before execution. Prefer backpressure to silently dropping receipts or retrying the same case in parallel. Include queue age in state only when it should legitimately change priority.

Separate provider retries from workflow retries. A transport retry repeats one logical decision; a workflow retry changes evidence or route. Give them different counters so an incident can reveal whether cost came from networking, uncertainty, or repeated generation.

IX.

CLAUDE, CODEX, GROK, AND ANY LLM

Route by capabilities, policy, and total economics rather than brand preference.

A. ONE DECISION LAYER, MANY GENERATORS

Claude, Codex, Grok, and other LLM products expose different capabilities, tool surfaces, context limits, prices, and deployment controls. A Jev layer should not hard-code brand preference into question wording. Build a provider registry in code, filter it by deterministic compatibility and policy, then let a typed Choice select among the remaining route classes.

The registry should describe what the current deployment can actually do: text generation, repository access, web access, structured outputs, tool support, data region, approved sensitivity class, expected latency, and current quota. Model names and terms change; capabilities are the stable interface.

B. NORMALIZE THE REQUEST

```
normalized = {
  "goal": goal,
  "input_kind": input_kind,
  "needs_web": needs_web,
  "needs_repo_write": needs_repo_write,
  "sensitivity": sensitivity,
  "latency_budget_ms": latency_budget,
  "allowed_routes": allowed_routes,
}
```

Ask Jev which allowed route class fits the normalized state. Code then resolves the class to a concrete provider and model version. This prevents question definitions from becoming obsolete every time a vendor renames a product.

C. PRESERVE PROVIDER-SPECIFIC STRENGTHS

A common adapter should normalize messages, tool proposals, citations, usage, and errors, but it should not erase unique capabilities. Keep provider-specific metadata available to policy and observability. The goal is portable orchestration, not the fiction that every model behaves identically.

H. DEFINE THE ADAPTER CONTRACT

At minimum, an adapter should return the serving model, normalized content, tool proposals, citations, finish reason, usage, latency, and provider error. It should preserve the raw response behind controlled access for debugging while exposing a stable typed shape to the rest of the harness.

Normalize cancellation, timeout, and partial-stream behavior. A provider that emitted half an answer before disconnecting should not look like a successful empty response. The postflight layer must know whether it is judging a complete artifact, a partial artifact, or no artifact at all.

D. ROUTE BY TOTAL ECONOMICS

FACTOR	WHY IT MATTERS
Context load	handoffs may resend the whole state
Cache	switching may lose a reusable prefix
Tool surface	provider may lack required action
Retry rate	cheap failures become expensive
Review	another model may reread everything
Trust	data policy can dominate price

Table VII. The cheapest token is not always the cheapest route.

E. USE THE SAME REVIEW CONTRACT

When possible, evaluate outputs from different LLMs with the same postflight question set and deterministic checks. This creates comparable outcome data. Keep model-specific exceptions explicit. If one provider supplies citations and another does not, the missing evidence should remain visible rather than being normalized away.

F. HANDLE PROVIDER FAILURE

Define whether a timeout may retry the same provider, fail over to another provider, use a smaller local model, or require human review. A fallback must pass the same data-egress and capability checks as the primary route. Never send sensitive state to a previously forbidden provider because the preferred API is unavailable.

G. AVOID ENDORSEMENT CLAIMS

Brand names on the cover illustrate interoperability, not a partnership or ranking. The architecture applies to any generative model whose output can be represented as state and whose call can be controlled by code. Provider choice should be based on your measured workload, permissions, and current commercial terms.

I. TEST PROVIDERS AGAINST THE SAME TASKS

Build a representative suite that includes success, ambiguity, missing tools, long context, sensitive state, and recoverable failure. Compare verified task outcomes rather than style preference. A shorter answer is not better if it omits evidence; a longer answer is not better if it increases review cost.

Re-run the suite when a provider changes a model, tool schema, or pricing tier. Keep route selection conservative until the new configuration has enough observations to estimate quality, latency, and fallback frequency.

X.

PRODUCTION SAFETY AND EVALUATION

Earn autonomy through staged rollout, calibrated thresholds, and replayable evidence.

A. SIDE EFFECTS REMAIN A POLICY PROBLEM

Jev can estimate whether a proposed action appears risky, relevant, or authorized by the supplied text. It cannot grant operating-system privileges, verify caller identity, or undo an action already sent to a provider. Put the decision call before the side effect, and enforce the final allow, ask, or deny result in code.

B. LAYER THE GATE

```
proposal
-> schema and type validation
-> identity and capability check
-> path / domain / amount allowlists
-> data-egress policy
-> Jev semantic review
-> risk-specific threshold
-> user or operator confirmation
-> execute with least privilege
-> immutable receipt
```

Ordering matters. If deterministic validation already fails, do not call Jev and do not let a favorable semantic answer override the failure. If the Jev service is unavailable, choose a documented fallback such as proposal-only or human review; do not silently treat absence as approval.

C. LAUNCH IN STAGES

STAGE	BEHAVIOR
Offline	replay labeled cases
Shadow	log decisions, change nothing
Assist	recommend routes to humans
Limited	automate low-risk slice
Expand	raise scope after evidence

I. PROTECT THE REVIEW PATH

Human review is a production dependency, not an infinite overflow queue. Estimate expected volume at each threshold, define the information shown to reviewers, and record the reviewer outcome. If the queue exceeds capacity, the safe response may be to reduce automation scope rather than lower the threshold.

Review interfaces should show the state fields and evidence that drove the decision, not only a model label. A reviewer must be able to correct the route, identify missing evidence, and mark the case for a new regression fixture.

D. EVALUATE OUTCOMES, NOT AGREEMENT

Model-to-model agreement is useful but not sufficient. Build a labeled set from real workflow outcomes and independent review. Measure each question, the composed policy, and the downstream task. A router can classify intent accurately yet still choose a worse provider because the capability registry or cost model is wrong.

E. WATCH THE RIGHT METRICS

Track coverage, automatic-action rate, human-review rate, false automation, false rejection, abstention, calibration by probability band, latency percentiles, provider errors, retry count, LLM cost, and verified task success. Segment by question version, state schema, model identifier, route, customer class, and risk level.

F. TEST JAGGED EDGES

Create cases with negation, quoted instructions, long lists, missing evidence, overlapping Choice options, mixed languages, adversarial content, and state that contains prompt injection. TypeSafe notes that Jev currently accepts text state and that English is its strongest language. Evaluate your own languages and domains before expanding automation.

G. MAKE ROLLBACK ORDINARY

Version every state builder, question set, threshold policy, provider registry, and fallback rule. A release should be reversible without changing application code in an emergency. Keep a kill switch that disables automatic actions while preserving observation and logging.

H. TURN FAILURES INTO ASSETS

Every incident should add at least one durable artifact: a regression case, a corrected question, a deterministic validator, a new no-match option, a tighter egress rule, or a revised threshold. The system improves when failures become executable knowledge rather than anecdotes in a postmortem.

J. ASSIGN OPERATIONAL OWNERSHIP

Name owners for the state builder, question contract, thresholds, provider registry, review queue, and incident response. A model integration without operational ownership decays as policies, traffic, and vendors change. The owner should have authority to disable automation without waiting for a model update.

Schedule periodic sampling even when dashboards look healthy. Aggregate accuracy can hide a failing language, tenant, route, or rare high-impact class. Review samples from every automated branch and every confidence band.

XI.

TEN-STEP PLAYBOOK AND SOURCES

A compact build order for taking the architecture from prototype to production.

A. THE TEN-STEP PRODUCTION PLAYBOOK

1. Draw the boundary. Use code for exact rules, Jev for bounded judgments, and an LLM for open-ended generation.
2. Build state. Send only the current evidence, name every field, and keep untrusted content separate from instructions.
3. Choose the primitive. Use Choice for a route, Score for an ordered rubric, and Noul for the probability of yes.
4. Decompose. Replace one broad prompt with independent atomic questions and compose their answers in code.
5. Batch. Ask all independent questions that share a state in one request, including inexpensive speculative questions.
6. Route before generation. Filter providers and tools by capability and policy, then use Jev to choose among allowed paths.
7. Verify after generation. Run deterministic checks first, then ask Jev about bounded semantic properties of the output.
8. Gate by risk. Translate distributions into automatic, review, and stop bands chosen from labeled data.
9. Record receipts. Bind state, questions, model, thresholds, route, output, and outcome to a versioned trace.
10. Roll out gradually. Begin offline, move to shadow and assist modes, automate a low-risk slice, and expand only with evidence.

B. PRE-LAUNCH CHECKLIST

Required fields validated; no-match options present; deterministic rules run before AI; secrets minimized; provider routes approved; failure behavior defined; thresholds evaluated on held-out cases; receipts replayable; rollback tested; human review staffed for the volume the policy will create.

C. THE FIRST WEEK

Run daily reviews of low-confidence cases, automated actions, fallbacks, and provider errors. Compare predicted routes with verified outcomes. Freeze question wording during the first observation window so changes in behavior can be attributed to traffic rather than to a moving contract.

At the end of the week, choose one correction with a measurable hypothesis: add a no-match option, remove a noisy field, change one threshold, or split one broad question. Replay the labeled set, deploy in shadow mode, and promote only when the new version improves the intended metric without shifting risk elsewhere.

D. SOURCE NOTES

The factual descriptions of Jev, System One, state, primitives, confidence, API shape, SDK installation, batching, guardrails, and coding-agent boundaries are based on TypeSafe's public documentation as accessed in September 2026. Product availability, model aliases, pricing, and performance claims can change; verify the live documentation before deployment.

TypeSafe AI, Introducing System One Models & Jev. <https://typesafe.ai/blog/introducing-system-one-models-and-jev>

TypeSafe Documentation, Quick start. <https://docs.typesafe.ai/introduction/quickstart>

TypeSafe Documentation, Jev with coding agents. <https://docs.typesafe.ai/introduction/coding-agents>

TypeSafe Documentation, State. <https://docs.typesafe.ai/concepts/state>

TypeSafe Documentation, Primitives. <https://docs.typesafe.ai/primitives>

TypeSafe Documentation, Confidence. <https://docs.typesafe.ai/confidence>

TypeSafe Documentation, How to build with TypeSafe. <https://docs.typesafe.ai/concepts/how-to-build-with-system-one>

TypeSafe Cookbook, Guardrails for LLMs. https://docs.typesafe.ai/cookbooks/llm_guardrails

TypeSafe Workflow Evals. <https://evals.typesafe.ai/>

E. EDITORIAL NOTE

This independent guide is educational and does not imply endorsement by TypeSafe, Anthropic, OpenAI, or xAI. Claude, Codex, and Grok are named only to explain a provider-neutral architecture. The cover diagram is an original schematic representation of the workflow described in this guide.

F. THE OPERATING PRINCIPLE

Do not ask one model to be the author, router, policy engine, judge, and auditor of its own work. Let the LLM generate. Let Jev answer narrow typed questions. Let deterministic code own authority. Let measured outcomes decide whether the system earns more autonomy.

G. WHAT SUCCESS LOOKS LIKE

A successful integration removes a repeated semantic decision from a generative transcript and turns it into a versioned contract. The system spends less on unnecessary generation, catches more unsupported outputs, explains uncertainty, and sends humans the cases where their attention changes the outcome.

The strongest signal is not the number of Jev calls. It is a higher verified completion rate with lower total latency, cost, and recovery work. If those outcomes do not improve, simplify the architecture or remove the decision layer from that branch.